@mention in textarea

Often asked at: Slack, Atlassian, LinkedIn
Why it matters: caret position, popover
Tech stack: React (Vite, JavaScript) · **Time limit:** 60 minutes

---

PART A: Thought process

These are the general steps for any machine coding task:

1. **Clarify**: restate the problem, ask questions, write down your assumptions.
2. **List the interactions**: every action the user can take and what should happen. This list is your checklist.
3. **Sketch the components**: who renders what, who owns which state.
4. **Design the state and data**: the minimum state, the data shape, and the API contract.
5. **Find the hard parts early**: name them before coding so they get time.
6. **Build in layers**: a working version first, then one feature at a time. Keep it runnable after every step.
7. **Cover the states**: loading, empty, error, plus edge cases.
8. **Wrap up**: say what you handled, the trade-offs, and what you'd add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, components, state design	8 min
Core build (textarea + trigger detection + list + insert)	20 min
Hard parts (caret position, keyboard, async search)	15 min
States, edge cases, accessibility	7 min
Wrap-up and talk through trade-offs	5 min

Think out loud. The interviewer grades how you reason as much as the code. Say why you choose something ("I'll anchor the popover at the `@`, not the caret, so it doesn't slide while typing"). If you are stuck, say what you're trying. A silent candidate with perfect code often scores lower than a talking candidate with a small bug.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A multi-line `<textarea>` where typing `@` opens a suggestion popover of users.
- Characters typed after `@` filter the list (`@al` → Alice, Alan).
- The popover appears **right next to the caret**, not at a fixed place.
- Pick a user with the mouse, or with ArrowUp / ArrowDown + Enter / Tab.
- Picking replaces `@al` with `@alice` and puts the caret right after it.
- Escape closes the popover. Moving the caret out of the `@token` or blurring closes it too.
- On submit, the comment is shown with mentions highlighted, and we know **which user ids** to notify.

Non-functional requirements

- **Performance**: debounce the search, cancel stale requests, do no work when the popover is closed.
- **Scalability**: the user list comes from an async API, not a hard-coded array, so it works with 100k users.
- **Error handling**: network failure shows a message and does not break typing.
- **Accessibility**: combobox pattern (`role="combobox"`, `aria-expanded`, `aria-activedescendant`, `role="listbox"` / `option`). Full keyboard use. Focus never leaves the textarea.
- **Reusability**: `MentionTextarea` is a controlled component (`value`, `onChange`, `onMention`), drop-in anywhere.
- **Extensibility**: the trigger rule and the search function are isolated, so adding `#channels` or `:emoji:` is a small change.

Clarifying questions to ask

1. Plain `<textarea>` or a rich editor (`contenteditable`)? Do mentions need to look highlighted *while typing*?
2. Where do users come from: a local list or a search API? How many users?
3. What does the backend want: plain text `@alice`, or a markup like `@[Alice](u1)`, or text + an array of ids?
4. When does `@` trigger: anywhere, or only at the start / after a space (so emails don't trigger it)?
5. If the user deletes part of a mention after picking it, is it still a mention?
6. Mobile / IME support needed (Chinese, Japanese input)?

Assumptions we build with

- Plain `<textarea>`; highlighting is shown in the posted comment, not live in the input.
- Users come from a mock async API with random latency (200-500 ms) that can fail.
- `@` triggers only at the start of the text or after whitespace. Handles are `\w+` (letters, digits, `_`).

- Output = plain text + array of mentioned users (ids). Only users **picked from the popover** count, and only if their `@handle` is still in the text.
- Max 8 suggestions shown.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type `@` at start or after a space	Popover opens under the `@` showing "Searching…", then users
Type `@al`	List filtered to handles starting with `al` first, then names containing it
Type `mail@gmail`	No popover (no space before `@`)
ArrowDown / ArrowUp	Highlight moves, wraps around, caret does **not** move
Enter or Tab with popover open	Selected user inserted as `@alice `, caret after the space, popover closes
Enter with popover closed	Normal newline
Click a suggestion	Same as Enter; textarea keeps focus
Hover a suggestion	It becomes the highlighted one
Escape	Popover closes, stays closed until a new `@`
Type a space after `@al`	Popover closes (token ended)
Click / arrow the caret into an existing `@tok`	Popover reopens for that token
Query with no match `@zzz`	"No users match @zzz"
API fails	"Couldn't load users. Keep typing to retry."
Typing fast `@a` → `@al` → `@ali`	Only the last request's results are shown
Caret near right edge / bottom of screen	Popover is clamped inside / flips above
Blur the textarea	Popover closes
Post	Comment rendered with `@alice` highlighted, "Notifies: Alice Johnson"

---

3. Design the architecture

Module	Responsibility
`App`	Owns the comment text, the picked users and the posted comments.

Module	Responsibility
`MentionTextarea`	Controlled textarea; detects the `@token`, keyboard nav, inserts the mention, positions the popover.
`MentionPopover`	Renders the list / loading / empty / error states. Pure UI.
`useMentionSearch`	Debounced, cancellable async search for a query.
`utils/mention.js`	Pure functions: `getMentionQuery`, `insertMention`, `tokenize`, `extractMentions`.
`utils/caretCoords.js`	Pixel position of a character inside a textarea (mirror div).
`api/searchUsers.js`	Mock API + pure `filterUsers` ranking.

```
App
 ├── <form>
 │    ├── MentionTextarea  ──uses──> useMentionSearch ──> api/searchUsers
 │    │     │                          utils/mention, utils/caretCoords
 │    │     └── MentionPopover (listbox)
 │    └── Post button
 └── Comment list (renders tokenize() parts)
```

Start simple: one component with a textarea and a filtered list from a local array. Split out the hook when the async search arrives, and the utils when the string logic gets tested on its own.

---

4. Design the state / data model

State or field	Owner	Purpose
`text`	App	The textarea value (controlled).
`picked`	App	Users chosen from the popover, used to decide what counts as a mention.
`comments`	App	Posted comments `{ id, parts, mentions }`.
`trigger`	MentionTextarea	`{ start, query }` of the `@token` at the caret, or `null`.
`pos`	MentionTextarea	Popover CSS position (`left` + `top` or `bottom`).
`activeIndex`	MentionTextarea	Which suggestion is highlighted.
`dismissedAt`	MentionTextarea	`start` of the token the user Escaped from.
`status`, `results`	useMentionSearch	`idle / loading / success / error` + the users.
`pendingCaret` (ref)	MentionTextarea	Caret position to restore after React re-renders the new value.

```
const [trigger, setTrigger] = useState(null);       // { start, query } | null
const [pos, setPos] = useState(null);               // { left, top } | { left, bottom }
const [activeIndex, setActiveIndex] = useState(0);
const [dismissedAt, setDismissedAt] = useState(null);
const pendingCaret = useRef(null);                  // not state: no re-render needed
```

Senior note: derive, don't store.

- `open` is derived: `trigger !== null && trigger.start !== dismissedAt`. No `isOpen` boolean to keep in sync.
- The query is derived from `text` + caret on every input/select; we never store a separate "query" string that could drift from the text.
- The final mentions are derived at submit by `tokenize(text, picked)`. If the user deletes `@alice` by hand, she is simply not in the result. No "remove mention" bookkeeping.
- `text` lives in the parent (controlled component) so the parent can clear it, save drafts, or submit it. The popover state is private to `MentionTextarea` because nobody else cares.

---

5. Design the data structures and contracts

Example data (stable ids, because a handle can be renamed, an id never changes; the backend notifies by id):

```
export const USERS = [
  { id: 'u1', name: 'Alice Johnson', handle: 'alice' },
  { id: 'u2', name: 'Alan Turing', handle: 'alan_t' },
  { id: 'u3', name: 'Bob Smith', handle: 'bob' },
  // ...
];
```

API contract (mocked)

```
searchUsers(query: string, { signal?: AbortSignal }) => Promise<User[]>
  input:   "al"
  output:  [{ id: "u1", name: "Alice Johnson", handle: "alice" },
            { id: "u2", name: "Alan Turing",   handle: "alan_t" }]
  errors:  rejects Error("Network error")  (query "fail" simulates it)
           rejects AbortError when signal aborts
  real API: GET /api/users/search?q=al&limit=8
```

Component contract

```
<MentionTextarea
  value={text}                 // string, controlled
  onChange={setText}           // (nextText: string) => void
  onMention={addPicked}        // (user: User) => void, fired on pick
  placeholder="..."
/>
```

Submit payload (what the backend gets):

```
{
  "text": "Hey @alice can you review?",
  "mentions": ["u1"]
}
```

Pure helpers

```
getMentionQuery("hi @al", 6)          -> { start: 3, query: "al" }
getMentionQuery("mail@gmail", 10)     -> null
insertMention("hi @al", 3, 6, "alice") -> { text: "hi @alice ", caret: 10 }
tokenize("hey @alice!", [alice])      -> [text "hey ", mention "@alice", text "!"]
```

---

6. Trace the data flow

Key interaction: the user types `l` so the text becomes `Hi @al`, then presses Enter.

```
keystroke "l"
   │
   ▼
textarea onChange ──> App.setText("Hi @al")
   │
   ▼
syncTrigger(): read el.selectionStart = 6
   getMentionQuery("Hi @al", 6) -> { start: 3, query: "al" }
   query changed -> setActiveIndex(0)
   computePosition(el, 3) -> mirror div measures the "@" -> { left: 30, top: 34 }
   │
   ▼
render: open = true -> useMentionSearch("al")
   effect: status = loading (old results kept), wait 150 ms debounce
   previous effect cleanup: clearTimeout + abort old request for "a"
   │
```

```
   ▼
searchUsers("al") resolves -> status = success, results = [Alice, Alan]
   │
   ▼
MentionPopover renders listbox at { left: 30, top: 34 }, Alice highlighted
   │
keystroke Enter
   ▼
onKeyDown: popover open + results -> preventDefault (no newline)
   select(Alice): insertMention("Hi @al", 3, 6, "alice") -> "Hi @alice ", caret 10
   pendingCaret = 10; onChange("Hi @alice "); onMention(Alice); setTrigger(null)
   │
   ▼
App: text = "Hi @alice ", picked = [Alice]
useLayoutEffect([value]): setSelectionRange(10, 10)  -> caret after the space
popover closed, useMentionSearch(null) -> idle, no request
```

---

7. Important technical challenges

Challenge 1: Where is the caret in pixels?

Problem. A `<textarea>` gives you `selectionStart` (a character index like `37`) but no x/y. The popover must appear under the `@` on line 3, after wrapping, after scrolling. There is no `getBoundingClientRect()` for a character inside a textarea.

Solution: the mirror div. Create an invisible `<div>` with the **same** font, padding, border, width, line-height and `white-space: pre-wrap` as the textarea. Put the text before the caret in it, then a `<span>` for the rest. The span's `offsetTop / offsetLeft` is exactly where the caret would be. Subtract `scrollTop` for a scrolled textarea. Anchor at the `@` index (not the live caret) so the popover doesn't slide right with every letter.

```
export function getCaretCoordinates(el, position) {
  const style = getComputedStyle(el);
  const div = document.createElement('div');
  PROPS.forEach((p) => { div.style[p] = style[p]; });       // font, padding, width…
  Object.assign(div.style, { position: 'absolute', visibility: 'hidden',
    whiteSpace: 'pre-wrap', overflowWrap: 'break-word' });
  div.textContent = el.value.slice(0, position);
  const span = document.createElement('span');
  span.textContent = el.value.slice(position) || '.';
  div.appendChild(span);
  document.body.appendChild(div);
  const top = span.offsetTop + parseFloat(style.borderTopWidth) - el.scrollTop;
```

```
  const left = span.offsetLeft + parseFloat(style.borderLeftWidth) - el.scrollLeft;
  document.body.removeChild(div);
  return { top, left, height: parseFloat(style.lineHeight) };
}
```

Then keep it on screen: clamp `left` so the popover doesn't overflow the right edge, and flip above the line when there isn't 240 px below.

```
const left = Math.max(0, Math.min(c.left, el.offsetWidth - POPOVER_W));
const roomBelow = window.innerHeight - (el.getBoundingClientRect().top + c.top + c.height);
return roomBelow >= POPOVER_H ? { left, top: c.top + c.height }
                              : { left, bottom: el.offsetHeight - c.top };
```

Challenge 2: Detecting the active @token (and only the right one)

Problem. The caret can be anywhere: `Hi @alice and @bo|b`. Emails (`mail@gmail.com`) must not trigger. Clicking back into an old token should reopen it. Picking while the caret is in the middle of `@al|xx` must replace the whole token.

Solution. Look only at the text **before the caret** and match a regex anchored at the end. The `(^|\s)` part rules out emails. Run it on `onChange` **and** `onSelect` (click, arrow keys) so caret moves are caught too.

```
const TRIGGER_RE = /(^|\s)@(\w{0,30})$/;
export function getMentionQuery(text, caret) {
  const match = TRIGGER_RE.exec(text.slice(0, caret));
  if (!match) return null;
  return { start: caret - match[2].length - 1, query: match[2] };
}

export function insertMention(text, start, caret, handle) {
  const tail = /^\w*/.exec(text.slice(caret))[0].length;   // rest of the word after caret
  const rest = text.slice(caret + tail);
  const mention = `@${handle}`;
  const sep = rest.startsWith(' ') ? '' : ' ';
  return { text: text.slice(0, start) + mention + sep + rest,
           caret: start + mention.length + 1 };
}
```

Challenge 3: Keyboard without losing the caret or the focus

Problem. ArrowDown normally moves the caret; Enter normally inserts a newline. Clicking an option normally blurs the textarea, which closes the popover **before** the click registers. And after we replace the value, React puts the caret at the end of the text.

Solution.

- In `onKeyDown`, only when the list is open: `preventDefault()` for arrows / Enter / Tab, and skip everything while an IME is composing (`e.nativeEvent.isComposing`).
- Use `onMouseDown` + `preventDefault()` on options so focus never leaves the textarea.
- Store the new caret in a ref and set it in `useLayoutEffect` after the value renders (before paint, so no flicker).

```
function onKeyDown(e) {
  if (!open || e.nativeEvent.isComposing) return;
  if (e.key === 'Escape') { e.preventDefault(); setDismissedAt(trigger.start); return; }
  if (!showList) return;
  const n = results.length;
  if (e.key === 'ArrowDown') { e.preventDefault(); setActiveIndex((i) => (i + 1) % n); }
  else if (e.key === 'ArrowUp') { e.preventDefault(); setActiveIndex((i) => (i - 1 + n) % n); }
  else if (e.key === 'Enter' || e.key === 'Tab') { e.preventDefault(); select(results[activeIndex]);
}
}

useLayoutEffect(() => {
  if (pendingCaret.current === null) return;
  ref.current.setSelectionRange(pendingCaret.current, pendingCaret.current);
  pendingCaret.current = null;
}, [value]);
```

Challenge 4: Async search with race conditions

Problem. The user types `@a`, `@al`, `@ali` quickly. Requests take 200-500 ms at random, so the response for `@a` can arrive **after** `@ali` and overwrite the correct list.

Solution. One effect per query: debounce 150 ms, and the cleanup aborts the previous timer **and** the in-flight request. A stale response can never call `setState`. Keep old results while loading so the list doesn't flash empty.

```
useEffect(() => {
  if (query === null) { setState({ status: 'idle', results: [] }); return; }
  const controller = new AbortController();
  setState((s) => ({ ...s, status: 'loading' }));
  const timer = setTimeout(() => {
    searchUsers(query, { signal: controller.signal })
      .then((results) => setState({ status: 'success', results }))
      .catch((err) => err.name !== 'AbortError' && setState({ status: 'error', results: [] }));
  }, delay);
```

```
  return () => { clearTimeout(timer); controller.abort(); };
}, [query, delay]);
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Vite app, `App` with a controlled textarea and Post button	You can type and post plain text
2	`getMentionQuery` + `onChange` / `onSelect` → `trigger` state	Log shows `{ start, query }` while typing `@al`
3	Static list from local `USERS` filtered by query, rendered under the textarea	A list appears and filters
4	`insertMention` + click to pick (mousedown) + caret restore	Clicking inserts `@alice` with the caret after it
5	Keyboard: arrows, Enter/Tab, Escape (`dismissedAt`), blur closes	Fully usable without the mouse
6	`getCaretCoordinates` + absolute popover anchored at the `@`	Popover follows the caret across lines
7	Clamp right edge + flip above + re-sync on scroll	Never off screen
8	Mock `searchUsers` + `useMentionSearch` (debounce + abort)	Async results, no races
9	Loading / empty / error messages in the popover	Every state is visible
10	`tokenize` + `extractMentions` , highlighted comments, "Notifies" line	Correct payload with ids
11	ARIA combobox attributes, `scrollIntoView` for the active option	Screen-reader and long-list friendly
12	Node assert self-check for the pure utils	Logic verified

---

9. Cover all states and edge cases

State / case	Handling
Initial	Empty textarea, Post disabled, "No comments yet."
Popover closed (`idle`)	No request sent, `aria-expanded="false"`
Loading, first time	"Searching…"
Loading, refining query	Previous results stay visible (no flicker)

State / case	Handling
Success	Up to 8 users, first one highlighted
Empty	`No users match "@zzz"`
Error	"Couldn't load users. Keep typing to retry." Next keystroke retries
Stale response	Aborted in effect cleanup; never rendered
`@` inside an email	Regex requires start or whitespace before `@`
`@` alone	Empty query shows the first 8 users
Space after `@al`	Token ends, popover closes
Escape	Closed until the caret leaves this token or a new `@` starts
Caret moved into an old `@tok`	`onSelect` re-detects and reopens
Text selected (range)	No trigger (`selectionStart !== selectionEnd`)
Pick with caret mid-token `@al	xx`
Pick before an existing space	No double space
Same user mentioned twice	Highlighted twice, notified once
Mention edited / deleted by hand	Not in `tokenize` output, so not notified
Handle typed but never picked	Plain text (no accidental mentions)
Multi-line text / wrapped lines	Mirror div uses `pre-wrap` + same width
Textarea scrolled	`- scrollTop` , re-sync on `scroll`
Near right edge / bottom	Clamp `left` , flip to `bottom`
IME composition	Keys ignored while `isComposing`
Blur	Popover closes; option clicks don't blur (mousedown)

---

10. Final structure

```
_76_mention_in_textarea/
├── index.html
├── package.json
├── vite.config.js
├── selfcheck.mjs             # node assert tests for the pure utils
└── src/
    ├── main.jsx
    ├── App.jsx               # form, picked users, posted comments
    ├── index.css
    ├── data/
```

```
    │   └── users.js           # mock users with stable ids
    ├── api/
    │   └── searchUsers.js     # filterUsers (pure) + abortable mock API
    ├── hooks/
    │   └── useMentionSearch.js
    ├── utils/
    │   ├── mention.js         # getMentionQuery, insertMention, tokenize, extractMentions
    │   └── caretCoords.js     # mirror-div caret position
    └── components/
        ├── MentionTextarea.jsx
        └── MentionPopover.jsx
```

---

11. Full working code

src/data/users.js

```
// Stable ids: a user can rename their handle, the id never changes.
export const USERS = [
  { id: 'u1', name: 'Alice Johnson', handle: 'alice' },
  { id: 'u2', name: 'Alan Turing', handle: 'alan_t' },
  { id: 'u3', name: 'Bob Smith', handle: 'bob' },
  { id: 'u4', name: 'Charlie Brown', handle: 'charlie' },
  { id: 'u5', name: 'Diana Prince', handle: 'diana' },
  { id: 'u6', name: 'Ethan Hunt', handle: 'ethan' },
  { id: 'u7', name: 'Fatima Khan', handle: 'fatima_k' },
  { id: 'u8', name: 'Grace Hopper', handle: 'grace' },
  { id: 'u9', name: 'Hiro Tanaka', handle: 'hiro' },
  { id: 'u10', name: 'Isha Verma', handle: 'isha' },
  { id: 'u11', name: 'Arjun Mehta', handle: 'arjun' },
  { id: 'u12', name: 'Priya Sharma', handle: 'priya' },
];
```

src/api/searchUsers.js

```
import { USERS } from '../data/users.js';

const MAX_RESULTS = 8;

// Pure ranking: handle prefix first, then name contains.
export function filterUsers(users, query) {
  const q = query.toLowerCase();
```

```
  const prefix = users.filter((u) => u.handle.toLowerCase().startsWith(q));
  const rest = users.filter((u) => !prefix.includes(u) && u.name.toLowerCase().includes(q));
  return [...prefix, ...rest].slice(0, MAX_RESULTS);
}

// Mock API: random latency (so responses can arrive out of order), abortable,
// and the query "fail" simulates a network error.
export function searchUsers(query, { signal } = {}) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      if (query === 'fail') reject(new Error('Network error'));
      else resolve(filterUsers(USERS, query));
    }, 200 + Math.random() * 300);

    signal?.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(new DOMException('Aborted', 'AbortError'));
    });
  });
}
```

src/utils/mention.js

```
// "@" only triggers at the start of the text or after whitespace,
// so "mail@gmail.com" never opens the popover.
const TRIGGER_RE = /(^|\s)@(\w{0,30})$/;

/** Returns { start, query } when the caret sits inside an @token, else null. */
export function getMentionQuery(text, caret) {
  const match = TRIGGER_RE.exec(text.slice(0, caret));
  if (!match) return null;
  const query = match[2];
  return { start: caret - query.length - 1, query };
}

/** Replaces the whole @token (including chars after the caret) with "@handle ". */
export function insertMention(text, start, caret, handle) {
  const tail = /^\w*/.exec(text.slice(caret))[0].length;
  const rest = text.slice(caret + tail);
  const mention = `@${handle}`;
  const sep = rest.startsWith(' ') ? '' : ' '; // never produce a double space
  return {
    text: text.slice(0, start) + mention + sep + rest,
```

```
    caret: start + mention.length + 1,
  };
}

/** Splits text into text / mention parts. Only handles the user actually picked count. */
export function tokenize(text, picked) {
  const byHandle = new Map(picked.map((u) => [u.handle.toLowerCase(), u]));
  const parts = [];
  let last = 0;

  for (const m of text.matchAll(/(^|\s)@(\w+)/g)) {
    const user = byHandle.get(m[2].toLowerCase());
    if (!user) continue;
    const at = m.index + m[1].length;
    if (at > last) parts.push({ type: 'text', value: text.slice(last, at) });
    parts.push({ type: 'mention', value: `@${m[2]}`, user });
    last = at + m[2].length + 1;
  }
  if (last < text.length) parts.push({ type: 'text', value: text.slice(last) });
  return parts;
}

/** Unique users still present in the text (a mention deleted by hand is dropped). */
export function extractMentions(parts) {
  const seen = new Map();
  parts.forEach((p) => p.type === 'mention' && seen.set(p.user.id, p.user));
  return [...seen.values()];
}
```

src/utils/caretCoords.js

```
// Styles that change where text wraps; the mirror must copy all of them.
const PROPS = [
  'boxSizing', 'width', 'height', 'borderTopWidth', 'borderRightWidth',
  'borderBottomWidth', 'borderLeftWidth', 'paddingTop', 'paddingRight',
  'paddingBottom', 'paddingLeft', 'fontStyle', 'fontVariant', 'fontWeight',
  'fontStretch', 'fontSize', 'lineHeight', 'fontFamily', 'textAlign',
  'textTransform', 'textIndent', 'letterSpacing', 'wordSpacing', 'tabSize',
];

/**
 * A textarea can't tell you where the caret is in pixels, so we build an
 * invisible div with identical styles, put the text before the caret in it,
```

```
 * and measure a <span> placed right at the caret.
 */
export function getCaretCoordinates(el, position) {
  const style = getComputedStyle(el);
  const div = document.createElement('div');
  PROPS.forEach((p) => { div.style[p] = style[p]; });
  Object.assign(div.style, {
    position: 'absolute', visibility: 'hidden', top: '0', left: '-9999px',
    whiteSpace: 'pre-wrap', overflowWrap: 'break-word',
    // a visible scrollbar narrows the text area, so the mirror needs one too
    overflowY: el.scrollHeight > el.clientHeight ? 'scroll' : 'hidden',
  });

  div.textContent = el.value.slice(0, position);
  const span = document.createElement('span');
  span.textContent = el.value.slice(position) || '.';
  div.appendChild(span);
  document.body.appendChild(div);

  const coords = {
    top: span.offsetTop + parseFloat(style.borderTopWidth) - el.scrollTop,
    left: span.offsetLeft + parseFloat(style.borderLeftWidth) - el.scrollLeft,
    height: parseFloat(style.lineHeight) || parseFloat(style.fontSize) * 1.2,
  };
  document.body.removeChild(div);
  return coords;
}
```

src/hooks/useMentionSearch.js

```
import { useEffect, useState } from 'react';
import { searchUsers } from '../api/searchUsers';

/** query === null means the popover is closed: no request. */
export function useMentionSearch(query, delay = 150) {
  const [state, setState] = useState({ status: 'idle', results: [] });

  useEffect(() => {
    if (query === null) {
      setState({ status: 'idle', results: [] });
      return;
    }
    const controller = new AbortController();
```

```
    // keep the old results while loading so the list doesn't flicker
    setState((s) => ({ ...s, status: 'loading' }));

    const timer = setTimeout(() => {
      searchUsers(query, { signal: controller.signal })
        .then((results) => setState({ status: 'success', results }))
        .catch((err) => {
          if (err.name !== 'AbortError') setState({ status: 'error', results: [] });
        });
    }, delay);

    // a newer keystroke cancels both the debounce and the in-flight request
    return () => {
      clearTimeout(timer);
      controller.abort();
    };
  }, [query, delay]);

  return state;
}
```

src/components/MentionTextarea.jsx

```
import { useId, useLayoutEffect, useRef, useState } from 'react';
import { getMentionQuery, insertMention } from '../utils/mention';
import { getCaretCoordinates } from '../utils/caretCoords';
import { useMentionSearch } from '../hooks/useMentionSearch';
import MentionPopover from './MentionPopover';

const POPOVER_W = 260;
const POPOVER_H = 240;

export default function MentionTextarea({ value, onChange, onMention, placeholder }) {
  const ref = useRef(null);
  const pendingCaret = useRef(null);
  const listId = useId();

  const [trigger, setTrigger] = useState(null); // { start, query } | null
  const [pos, setPos] = useState(null); // popover style
  const [activeIndex, setActiveIndex] = useState(0);
  const [dismissedAt, setDismissedAt] = useState(null); // Esc closes until a new "@"

  const open = trigger !== null && trigger.start !== dismissedAt;
```

```
  const { status, results } = useMentionSearch(open ? trigger.query : null);
  const showList = open && results.length > 0 && status !== 'error';

  // After inserting a mention React re-renders the value; put the caret after it.
  useLayoutEffect(() => {
    if (pendingCaret.current === null) return;
    ref.current.setSelectionRange(pendingCaret.current, pendingCaret.current);
    pendingCaret.current = null;
  }, [value]);

  // Re-read caret on every input, click, arrow move and scroll.
  function syncTrigger() {
    const el = ref.current;
    const next = el.selectionStart === el.selectionEnd ? getMentionQuery(el.value,
el.selectionStart) : null;
    if (next?.query !== trigger?.query || next?.start !== trigger?.start) setActiveIndex(0);
    if (!next) setDismissedAt(null);
    setTrigger(next);
    if (next) setPos(computePosition(el, next.start));
  }

  function select(user) {
    const el = ref.current;
    const result = insertMention(value, trigger.start, el.selectionStart, user.handle);
    pendingCaret.current = result.caret;
    onChange(result.text);
    onMention?.(user);
    setTrigger(null);
  }

  function onKeyDown(e) {
    if (!open || e.nativeEvent.isComposing) return; // don't hijack keys during IME input
    if (e.key === 'Escape') {
      e.preventDefault();
      setDismissedAt(trigger.start);
      return;
    }
    if (!showList) return;
    const n = results.length;
    if (e.key === 'ArrowDown') { e.preventDefault(); setActiveIndex((i) => (i + 1) % n); }
    else if (e.key === 'ArrowUp') { e.preventDefault(); setActiveIndex((i) => (i - 1 + n) % n); }
    else if (e.key === 'Enter' || e.key === 'Tab') { e.preventDefault();
select(results[activeIndex]); }
  }
```

```
  return (
    <div className="mention-box">
      <textarea
        ref={ref}
        value={value}
        placeholder={placeholder}
        rows={4}
        role="combobox"
        aria-autocomplete="list"
        aria-expanded={showList}
        aria-controls={showList ? listId : undefined}
        aria-activedescendant={showList ? `${listId}-${results[activeIndex]?.id}` : undefined}
        onChange={(e) => { onChange(e.target.value); syncTrigger(); }}
        onSelect={syncTrigger}
        onScroll={() => open && syncTrigger()}
        onKeyDown={onKeyDown}
        onBlur={() => setTrigger(null)}
      />
      {open && pos && (
        <MentionPopover
          id={listId}
          style={pos}
          query={trigger.query}
          status={status}
          results={results}
          activeIndex={activeIndex}
          onSelect={select}
          onHover={setActiveIndex}
        />
      )}
    </div>
  );
}

// Anchor at the "@" (not the caret) so the popover doesn't slide while typing.
// Clamp to the right edge, flip above when there's no room below.
function computePosition(el, atIndex) {
  const c = getCaretCoordinates(el, atIndex);
  const left = Math.max(0, Math.min(c.left, el.offsetWidth - POPOVER_W));
  const roomBelow = window.innerHeight - (el.getBoundingClientRect().top + c.top + c.height);
  return roomBelow >= POPOVER_H
    ? { left, top: c.top + c.height }
```

```
    : { left, bottom: el.offsetHeight - c.top };
}
```

src/components/MentionPopover.jsx

```
import { useEffect, useRef } from 'react';

export default function MentionPopover({ id, style, query, status, results, activeIndex, onSelect,
onHover }) {
  const listRef = useRef(null);

  // keep the keyboard-highlighted option visible inside the scrollable list
  useEffect(() => {
    listRef.current?.querySelector('[aria-selected="true"]')?.scrollIntoView({ block: 'nearest' });
  }, [activeIndex, results]);

  let message = null;
  if (status === 'error') message = "Couldn't load users. Keep typing to retry.";
  else if (results.length === 0 && status === 'loading') message = 'Searching…';
  else if (results.length === 0 && status === 'success') message = `No users match "@${query}"`;

  return (
    <div className="popover" style={style}>
      {message ? (
        <p className="popover-msg" role="status">{message}</p>
      ) : (
        <ul id={id} role="listbox" ref={listRef} aria-label="Mention suggestions">
          {results.map((user, i) => (
            <li
              key={user.id}
              id={`${id}-${user.id}`}
              role="option"
              aria-selected={i === activeIndex}
              className={i === activeIndex ? 'option active' : 'option'}
              // mousedown + preventDefault keeps focus (and the caret) in the textarea
              onMouseDown={(e) => { e.preventDefault(); onSelect(user); }}
              onMouseEnter={() => onHover(i)}
            >
              <span className="avatar">{user.name[0]}</span>
              <span className="name">{user.name}</span>
              <span className="handle">@{user.handle}</span>
            </li>
          ))}
```

```
        </ul>
      )}
    </div>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import MentionTextarea from './components/MentionTextarea';
import { extractMentions, tokenize } from './utils/mention';

export default function App() {
  const [text, setText] = useState('');
  const [picked, setPicked] = useState([]); // users chosen from the popover
  const [comments, setComments] = useState([]);

  const addPicked = (user) =>
    setPicked((list) => (list.some((u) => u.id === user.id) ? list : [...list, user]));

  function submit(e) {
    e.preventDefault();
    const body = text.trim();
    if (!body) return;
    const parts = tokenize(body, picked);
    // payload the backend would get: plain text + the ids to notify
    setComments((list) => [{ id: crypto.randomUUID(), parts, mentions: extractMentions(parts) },
...list]);
    setText('');
    setPicked([]);
  }

  return (
    <main className="app">
      <h1>@mention in textarea</h1>
      <form onSubmit={submit}>
        <MentionTextarea
          value={text}
          onChange={setText}
          onMention={addPicked}
          placeholder="Write a comment… type @ to mention someone"
        />
        <button type="submit" disabled={!text.trim()}>Post</button>
```

```
      </form>

      {comments.length === 0 ? (
        <p className="empty">No comments yet.</p>
      ) : (
        <ul className="comments">
          {comments.map((c) => (
            <li key={c.id}>
              <p>
                {c.parts.map((p, i) =>
                  p.type === 'mention'
                    ? <span key={i} className="mention" title={p.user.name}>{p.value}</span>
                    : <span key={i}>{p.value}</span>,
                )}
              </p>
              {c.mentions.length > 0 && (
                <small>Notifies: {c.mentions.map((u) => u.name).join(', ')}</small>
              )}
            </li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/index.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f5f6f8; color: #1d1f23; }
```

```
.app { max-width: 640px; margin: 40px auto; padding: 0 16px; }

form { display: flex; flex-direction: column; gap: 8px; }
.mention-box { position: relative; }
textarea {
  display: block; width: 100%; padding: 10px 12px; resize: vertical;
  font: inherit; font-size: 15px; line-height: 1.5;
  border: 1px solid #c9ccd1; border-radius: 8px; background: #fff;
}
textarea:focus { outline: 2px solid #4f6bed; border-color: transparent; }
button { align-self: flex-end; padding: 8px 18px; border: 0; border-radius: 6px; background:
#4f6bed; color: #fff; font: inherit; cursor: pointer; }
button:disabled { opacity: 0.5; cursor: not-allowed; }

.popover {
  position: absolute; z-index: 10; width: 260px; max-height: 240px; overflow-y: auto;
  background: #fff; border: 1px solid #dfe1e5; border-radius: 8px; box-shadow: 0 8px 24px rgb(0 0 0
/ 0.12);
}
.popover ul { list-style: none; margin: 0; padding: 4px; }
.popover-msg { margin: 0; padding: 12px; color: #666; font-size: 14px; }
.option { display: flex; align-items: center; gap: 8px; padding: 6px 8px; border-radius: 6px;
cursor: pointer; }
.option.active { background: #e8ecfd; }
.avatar { width: 26px; height: 26px; border-radius: 50%; display: grid; place-items: center;
background: #4f6bed; color: #fff; font-size: 13px; flex: none; }
.name { font-size: 14px; }
.handle { margin-left: auto; color: #777; font-size: 13px; }

.comments { list-style: none; padding: 0; margin-top: 24px; display: grid; gap: 10px; }
.comments li { background: #fff; border-radius: 8px; padding: 12px 14px; }
.comments p { margin: 0 0 4px; white-space: pre-wrap; }
.comments small { color: #777; }
.mention { color: #4f6bed; background: #e8ecfd; border-radius: 4px; padding: 0 2px; font-weight:
500; }
.empty { color: #777; margin-top: 24px; }
```

`selfcheck.mjs` (run with `node selfcheck.mjs`)

```
import assert from 'node:assert/strict';
import { getMentionQuery, insertMention, tokenize, extractMentions } from './src/utils/mention.js';
import { filterUsers } from './src/api/searchUsers.js';
import { USERS } from './src/data/users.js';
```

```
// trigger detection
assert.deepEqual(getMentionQuery('@', 1), { start: 0, query: '' });
assert.deepEqual(getMentionQuery('hi @al', 6), { start: 3, query: 'al' });
assert.deepEqual(getMentionQuery('hi @al', 5), { start: 3, query: 'a' });
assert.equal(getMentionQuery('mail@gmail', 10), null);
assert.equal(getMentionQuery('@al ice', 7), null);
assert.deepEqual(getMentionQuery('a\n@bo', 5), { start: 2, query: 'bo' });
assert.deepEqual(getMentionQuery('@a @b', 5), { start: 3, query: 'b' });

// insertion replaces the whole token, even past the caret
assert.deepEqual(insertMention('hi @al', 3, 6, 'alice'), { text: 'hi @alice ', caret: 10 });
assert.deepEqual(insertMention('hi @alxx there', 3, 6, 'alice'), { text: 'hi @alice there', caret:
10 });

// tokenize only counts picked users
const alice = USERS[0];
const parts = tokenize('hey @alice and @bob, @alice!', [alice]);
assert.deepEqual(parts.map((p) => p.type), ['text', 'mention', 'text', 'mention', 'text']);
assert.equal(parts[1].value, '@alice');
assert.equal(parts.length, 5); assert.equal(parts[2].value, ' and @bob, ');
const twice = tokenize('@alice @alice', [alice]);
assert.equal(twice.filter((p) => p.type === 'mention').length, 2);
assert.deepEqual(extractMentions(twice), [alice]);
assert.deepEqual(tokenize('plain', []), [{ type: 'text', value: 'plain' }]);

// search ranking: handle prefix first
assert.deepEqual(filterUsers(USERS, 'al').map((u) => u.handle).slice(0, 2), ['alice', 'alan_t']);
assert.equal(filterUsers(USERS, '').length, 8);
assert.equal(filterUsers(USERS, 'zzz').length, 0);
console.log('selfcheck ok');
```

---

12. Wrap-up

Edge cases handled

- Emails don't trigger; `@` alone lists users; space ends the token.
- Caret moved by click or arrows reopens the right token; range selections don't trigger.
- Mid-token pick replaces the whole word; no double spaces; caret lands after the mention.
- Escape, blur, IME composition, right edge / bottom flip, scrolled and wrapped text.
- Out-of-order responses aborted; loading / empty / error states shown.
- Only picked users count, deleted mentions are dropped, duplicates notified once.

Trade-offs to mention

- **Plain textarea vs `contenteditable`**: textarea is robust (undo, paste, mobile keyboards, forms) but cannot style text inside it, so no live chip highlighting. `contenteditable` (Slack, Notion, Lexical, Slate) can, at the cost of much harder caret, paste and undo handling.
- **Plain `@handle` text vs markup `@[Alice](u1)`**: plain text is readable and simple, but a renamed handle or two users with similar handles is ambiguous. Markup or offsets with ids are robust but need parsing for display and editing.
- **Mirror div**: accurate and dependency-free, but it measures on each caret change (cheap: one hidden node). It must copy every style that affects wrapping.
- **Anchoring at `@`** instead of the live caret: stable popover, slightly less "following" feel.
- **Debounce 150 ms**: fewer requests vs a little latency. A local list would skip the debounce.

What to add with more time

- Live highlight in the input with a **backdrop overlay** (a div behind a transparent textarea showing the same text with `<mark>` around mentions).
- Atomic mentions: Backspace on `@alice` deletes the whole mention.
- Result caching per query (Map / LRU), prefetch on `@`.
- Portal + Floating UI for the popover so it escapes `overflow: hidden` parents; reposition on window resize.
- Unit tests with Vitest + React Testing Library (keyboard flow, race condition with fake timers), Playwright for caret position across browsers.
- Multiple triggers via config: `{ char: '#', search: searchChannels }`.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why only trigger `@` after whitespace or at the start?

Otherwise every email address (`me@site.com`) would open the popover. Slack, GitHub and Jira all use this rule. The regex `(^|\s)@` makes it one line, and it's easy to relax if the product wants it.

Q2. What should happen if the user types `@alice` by hand without picking from the list?

In our version it stays plain text: only users chosen from the popover are mentions. That avoids accidental notifications and ambiguity. If the product wants it, on submit we can resolve unknown handles with one batch API call (`/users?handles=alice,bob`).

Q3. Why a plain textarea and not `contenteditable`?

Textarea gives undo/redo, paste sanitising, spellcheck, mobile keyboards, form integration and accessibility for free. `contenteditable` is needed only if mentions must render as chips while typing. In

an interview, textarea gets a correct result inside 60 minutes; I'd mention Lexical / Slate / TipTap for production rich text.

State design

Q4. Why not store an `isOpen` boolean?

It would duplicate information. `open` is fully determined by `trigger` and `dismissedAt`, so deriving it means there's no way for the two to get out of sync (e.g. open with no query).

Q5. Why is `pendingCaret` a ref and not state?

It's a one-off instruction for the next layout effect, not something the UI renders. State would cause an extra render for nothing. A ref is read and cleared inside the same commit.

Q6. Who should own the `text` state?

The parent: it needs to submit, clear, save drafts or prefill a reply. The popover state (trigger, active index, position) stays inside `MentionTextarea` since no parent cares about it. That is the controlled-component split.

Challenge 1: caret position

Q7. How do you get the pixel position of the caret in a textarea?

There is no API for it. Use a hidden mirror div with the same font, padding, width, border, line-height and `white-space: pre-wrap`, fill it with the text before the caret plus a span, and read the span's `offsetTop / offsetLeft`, minus the textarea's `scrollTop`.

Q8. What if the textarea is inside a scrolling container or a modal with `overflow: hidden`?

An absolutely positioned popover would be clipped. Render it in a portal on `document.body` with `position: fixed`, using `textarea.getBoundingClientRect()` plus the mirror coordinates, and recompute on scroll / resize (or use Floating UI's `autoUpdate`).

Q9. Why does the mirror div need `overflowY: scroll` sometimes?

When the textarea shows a scrollbar, its text area is ~15 px narrower, so lines wrap earlier. If the mirror has no scrollbar, its lines wrap later and the measured caret lands on the wrong line.

Challenge 2: token detection

Q10. Why run detection on `onSelect` as well as `onChange`?

`onChange` only fires when text changes. Clicking into `@bo` or moving with the arrow keys changes the caret without changing the text, and the popover should react to that too.

Q11. How would you support handles with spaces, like "Alice Johnson"?

Allow spaces in the query (`[\w ]{0,30}`), but then "token ended" can't mean "space typed". Close instead when the query has no results for, say, two words, or on Escape. Insert the display name and store the id in markup, since the text alone becomes ambiguous.

Challenge 3: keyboard and focus

Q12. Why `onMouseDown` + `preventDefault` instead of `onClick` on options?

The order is mousedown → blur → mouseup → click. With `onClick`, the textarea's blur closes the popover first and the click lands on nothing. Preventing mousedown's default keeps focus in the textarea, so the caret and the selection range are still valid.

Q13. What breaks with IME (Japanese/Chinese) input?

While composing, Enter confirms the composition, not our suggestion. We check `e.nativeEvent.isComposing` and ignore keys during composition, otherwise users can't type in those languages.

Challenge 4: async search

Q14. Debounce or throttle for the search?

Debounce: we only care about the query once the user pauses. Throttle would send requests for intermediate queries we will throw away. Combined with `AbortController` there are no wasted renders even if a request slips through.

Q15. What if the API is slow (2-3 seconds)?

Keep showing the previous results with a subtle loading indicator, cache results per query so Backspace is instant, and prefetch a default list (recent collaborators) the moment `@` is typed so the popover is never empty.

Q16. What if there are 1 million users?

Never send them to the client. Search on the server with a prefix index (trie / Elasticsearch edge n-grams), return the top 8 ranked by relevance (people in this channel, recent interactions). On the client, cap results and cache. Rendering 8 items needs no virtualization.

Implementation and output

Q17. How do you send mentions to the backend?

Plain text plus an array of user ids, or a markup string like `Hey @[Alice](u1)`. Ids are required because handles can change. The server re-validates the ids (the user exists and can see this channel) before sending notifications.

Q18. How would you show mentions highlighted while typing in a textarea?

The backdrop technique: a div behind the textarea with identical styles renders the same text with `<mark>` around mentions; the textarea's text is made transparent (`color: transparent; caret-color: black`) and its scroll is synced to the backdrop.

Q19. How would you add `#channels` and `:emoji:` triggers?

Make the trigger configurable: `triggers = [{ char: '@', search: searchUsers }, { char: '#', search: searchChannels }]` , build the regex from the chars, and pass the matched trigger to the hook. The rest (position, keyboard, insert) is identical.

Performance and trade-offs

Q20. Isn't creating a mirror div on every keystroke slow?

It's one hidden element and one layout read per keystroke while the popover is open, well under a millisecond. If profiling showed a problem, keep one mirror div alive and only update its text. It never runs when the popover is closed.

Q21. Why not use a library like `react-mentions` ?

In production I'd consider it, or a rich-text editor's mention plugin. In an interview the point is to show the hard parts: caret position, focus handling, races and accessibility. I can also list what I'd check in a library: bundle size, IME support, a11y.

Q22. How do you test this?

Pure functions (`getMentionQuery` , `insertMention` , `tokenize`) with unit tests. Component tests with Testing Library: type `@al` , assert listbox options, ArrowDown + Enter, assert value and caret. Use fake timers to fire responses out of order for the race test. Caret pixel position belongs in a real browser test (Playwright).